Joint Service Caching, Communication and Computing Resource Allocation in Collaborative MEC Systems: A DRL-Based Two-Timescale Approach

Qianqian Liu, Haixia Zhang, *Senior Member, IEEE*, Xin Zhang, and Dongfeng Yuan, *Senior Member, IEEE*

***Abstract*— Meeting the strict Quality of Service (QoS) requirements of terminals has imposed a significant challenge on Multi-access Edge Computing (MEC) systems, due to the limited multi-dimensional resources. To address this challenge, we propose a collaborative MEC framework that facilitates resource sharing between the edge servers, and with the aim to maximize the long-term QoS and reduce the cache switching costs through joint optimization of service caching, collaborative offloading, and computation and communication resource allocation. The dual timescale feature and temporal recurrence relationship between service caching and other resource allocation make solving the problem even more challenging. To solve it, we propose a deep reinforcement learning (DRL)-based dual timescale scheme, called DGL-DDPG, which is composed of a short-term genetic algorithm (GA) and a long short-term memory network-based deep deterministic policy gradient (LSTM-DDPG). In doing so, we reformulate the optimization problem as a Markov decision process (MDP) where the small-timescale resource allocation decisions generated by an improved GA are taken as the states and inputted into a centralized LSTM-DDPG agent to generate the service caching decision for the large-timescale. Simulation results demonstrate that the proposed algorithm outperforms the baseline algorithms in terms of the average QoS and the cache switching costs.**

***Index Terms*— Multi-dimensional resources allocation, collaborative offloading, service caching, two-timescale, long short-term memory (LSTM) network, deep deterministic policy gradient.**

I. Introduction

THE evolution of the fifth generation mobile communication (5G), artificial intelligence (AI) and Internet of Things (IoT) has facilitated the emergence of various intelligent applications in Industrial Internet of Things (IIoT) systems. Such are normally delay-sensitive and computation-intensive tasks, like AI-driven remote equipment monitoring, predictive maintenance, and quality control, which also have strict Quality of Service (QoS) requirements in terms of delay and energy consumption [1], [2]. Considering that the time-varying sensing data and the low-power nature of devices in IIoT environments, terminal devices (TDs) may not be able to support these demanding tasks alone. So, splitting and offloading tasks to the edge servers (ESs) is an alternative [3], [4].

With multi-access Edge Computing (MEC), the computing, storage, and intelligence resources can be sunk to the ESs, making the resources close to the TDs, thus can effectively reduce end-to-end delay and energy consumption needed for task processing [5]. A lot of works have focused on how to schedule MEC's limited resources to meet the strict QoS requirements of the tasks. Most of works tackle this problem by optimizing the communication and computing resources to reduce delay and energy consumption [6], [7], [8], [9]. However, these studies have typically overlooked the fact that processing tasks offloaded from TDs may require specific service model stored at ESs ahead, including task-related machine learning models and databases known as edge service caching [10]. The edge service caching related to the short-term offloading and resource allocation, and the offloading strategy also depends on the service caching [11], [12], [13], [14]. Therefore, scheduling of multi-dimensional resources, i.e., the communication computing and caching resources, should be done jointly at the ES, although they are in different timescales.

Studies in [15], [16], and [17] have investigated jointly allocating service caching, communication and computing resources in MEC networks. However, these works are done based on an unrealistic assumption that service caching and other resource allocation decisions can be made simultaneously. From the practical perspective, caching is a decision-making process based on statistical information over a large timescale, with decision-making intervals in days and hours. On the other hand, computing and communication involves instantaneous decisions made in seconds or minutes. If we optimize the multi-dimensional resources together on a small timescale, as in [15], [16], and [17], will cause

Manuscript received 13 July 2023; revised 16 April 2024 and 12 July 2024; accepted 14 July 2024. Date of publication 25 July 2024; date of current version 11 October 2024. This work was supported in part by the Joint Funds of the National Natural Science Foundation of China (NSFC) under Grant U22A2003 and Grant U23A20277 and in part by NSFC under Grant 62271288. The associate editor coordinating the review of this article and approving it for publication was X. Cao. *(Corresponding author: Haixia Zhang.)*

Qianqian Liu, Haixia Zhang, and Xin Zhang are with Shandong Key Laboratory of Intelligent Communication and Sensing-Computing Integration, Jinan 250061, China, and also with the School of Control Science and Engineering, Shandong University, Jinan 250061, China (e-mail: liuqiansdu@mail.sdu.edu.cn; haixia.zhang@sdu.edu.cn; xin.zhang0892@mail.sdu.edu.cn).

Dongfeng Yuan is with Shandong Key Laboratory of Intelligent Communication and Sensing-Computing Integration, Jinan 250061, China (e-mail: dfyuan@sdu.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TWC.2024.3430486.

Digital Object Identifier 10.1109/TWC.2024.3430486

1536-1276 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Univ of Science and Tech Beijing. Downloaded on June 09,2026 at 10:45:34 UTC from IEEE Xplore. Restrictions apply.

frequent cache switching and huge communication overheads. Therefore, for practical concern, the two should be jointly optimized over different timescales.

To address this, in this work we consider a MEC-enabled IIoT system with the goal to improve the long-term QoS and reduce the cache switching costs. Achieving this requires a reasonable resource allocation strategy for both large and small timescales. If we only rely on a given service caching strategy (large timescale), more than one ESs may cache the same service model. This will fail to utilize the storage diversity among ESs. Besides, for the case that there is no service model at an ES, the tasks have to be offloaded to the other ES or CS with the needed service model for further processing. Such forward transmissions and results return will not only increase the communication overhead of the ES, but may also waste the computing resources of the ES, and finally increase the task processing delay. On the contrary, if multiple ESs work together to cache the service models and handle the computation tasks cooperatively, unnecessary transmission (tasks and service models) may be greatly reduced, and accordingly both TD's QoS and resource utilization can be improved [18], [7], [19], [20], [21], [22].

To the best knowledge of ours, few literature has addressed this in MEC-enabled IIoT systems. To fill this research gap, we investigate two-timescale multi-dimensional resources allocation with ESs collaboration to improve the long-term QoS and reduce the cache switching costs by jointly optimizing the long-term service caching, the short-term collaborative offloading, as well as the computing and bandwidth allocation. It is a typical two-timescale optimization problem. How to handle the coupled multi-dimensional resources to realize the joint optimization of large and small timescales is the biggest challenge. Together with the collaboration among ESs make the problem even difficult to solve. To solve it, we utilize deep reinforcement learning (DRL) for the long-term service caching strategy, which is a powerful model-free learning method and is suitable for handling complex long-term sequential decision-making problems [9], [15], [16], [23]. Additionally, we use genetic algorithm (GA) to solve cooperative resource scheduling problems on small timescale, since it is easy to embed in DRL to enhance the training effect [24]. More importantly, the long short-term memory (LSTM) network is used as a bridge between the large and small timescales, leveraging its temporal processing and prediction capabilities to capture temporal relationships between the long-term service caching and the short-term offloading and resource allocation [25], [26], [27], [28], [29]. Therefore, a two-timescale scheme based on GA, LSTM and DRL called DGL-DDPG is developed to address the joint cross-timescale multi-dimensional resources optimization problem. For the large timescale, we propose a LSTM-assisted deep deterministic policy gradient (LSTM-DDPG) algorithm to generate service caching strategies. Meanwhile, for the small timescale, we propose an improved genetic algorithm (Improved-GA) to generate immediate decisions for collaborative offloading, computing, and bandwidth resource allocation. The main contributions of the paper can be summarized as follows:

1) We establish a collaborative MEC-enabled IIoT system architecture in which the ESs collaborate with each other to cache service models, and handle computational tasks. We propose a two-timescale multi-dimensional resources scheduling scheme, named as DGL-DDPG,[1] to jointly optimize the long-term service caching, short-term collaborative offloading, computing, and bandwidth resource allocation for the established system to maximize the long-term QoS of all TDs and reduce the long-term cache switching costs of all ESs.
2) DGL-DDPG can intelligently and dynamically schedule multi-dimensional resources across timescales with the assistance of the LSTM-DDPG and the Improved-GA. In doing so, we transform multi-dimensional resource allocations at different timescales into a Markov Decision Process (MDP), where instantaneous resource allocation decisions generated by Improved-GA are treated as states. By feeding these states into a centralized LSTM-DDPG agent, the long-term service caching decisions are generated.
3) The LSTM-DDPG based service caching algorithm can model the temporal relationship between the long-term service caching and the short-term offloading and predict service demand variance, and make a long-term optimal service caching decision. Furthermore, we design a hybrid coding based Improved-GA to generate both discrete offloading decisions and continuous allocation of computing and bandwidth resources at small-timescale.

The remainder of this paper is organized as follows. Section II presents the related work. Section III describes the system model and problem formulation. Section IV elaborates on the DRL-based two timescale optimization algorithm. Section V presents simulation results that evaluate the convergence and performance of the proposed algorithms. Finally, Section VI presents the conclusions drawn from the study.

II. Related Work

This section provides a review of recent studies on resource allocation in MEC systems, categorized based on joint allocation of communication and computing resources, joint allocation of service caching and computing resources, and joint allocation of service caching, communication and computing resources.

A. Joint Communication and Computing Resources Allocation

For the allocation of communication and computing resources in MEC networks, most researcher focus on reducing the task processing delay and/or energy consumption through aspects such as task offloading (partially), computing resource allocation, power control, and channel selection.

Tan et al. [30] proposed a two-level alternation method to minimize total energy consumption in multi-user collaborative MEC networks. The method uses a heuristic algorithm in the upper level for collaboration and offloading decisions

[1] https://github.com/liuqiansdu/Two-timescale-DGL-DDPG-.git

Authorized licensed use limited to: Univ of Science and Tech Beijing. Downloaded on June 09,2026 at 10:45:34 UTC from IEEE Xplore. Restrictions apply.

and a DRL-based algorithm in the lower level for power, subcarrier, and computing resource allocation. Kai et al. [18] proposed a pipeline-based offloading scheme to minimize the overall latency for all users in a cloud-edge-end collaborative MEC network. They utilized the classic successive convex approximation (SCA) approach to optimize the offloading strategy, computation resource allocation, delivery rates, and power allocation. In [31], the authors considered the increasing tasks with stringent QoS requirements, as well as the limited communication and computational resources in multi-user multi-edge MEC networks. They formulated a long-term throughput maximization problem from the perspective of service providers under latency constraints. However, the above literature does not take into account the impact of task-related service caching. They all take it for granted that all the needed services are cached at all ESs. This is not realistic for the ESs with limited cache resources.

B. Joint Service Caching and Computing Resources Allocation

Works on service caching configuration and computing offloading mainly aim to reduce the task processing delay and energy consumption by jointly scheduling the services models, offloaded tasks, and computing resources within a small-timescale slot.

With the aim of maximizing the users' QoS, Chu et al. [32] proposed a Lagrangian-dual based distributed algorithm to optimize the service selection and computing resource allocation for a single MEC network. They further designed an online service caching update strategy based on the cost of services switching. Similarly, Zhou et al. [33] extended Chu's optimization problem to a multi-ES collaborative MEC system and proposed a DDPG-based algorithm to minimize the long-term energy consumption. Zhou et al. [34] investigated a use case of reverse computation offloading and service caching in a three-tier MEC system, in which the cloud server can purchase computation and communication resources from the ESs to assist with computing the offloaded tasks. Generally, the caching resources are adjusted at a larger timescale than that the computation resources are done at. However, this feature has not been captured in the literature mentioned above. For this, Zhang et al. [35] proposed a hierarchical DDPG-based algorithm to deal with the dual timescale resource allocation for IoT systems to minimize the delay and energy consumption. Lyapunov optimization approach and dependent rounding technique are adopted to solve the problem of the long-term service caching and the short-term UAV trajectory, user association, and offloading decisions optimization, respectively [36]. However, the aforementioned literature has not taken into account the communication resource needed in the caching and offloading process.

C. Joint Service Caching, Communication and Computing Resources Allocation

For the joint multi-dimensional resources optimization in MEC networks, most literature mainly studies service caching configuration, task offloading, and computing and communication resources allocation at the same timescale, with the goal of reducing the task processing delay, energy consumption, and resource purchase costs.

Bi et al. [37] have investigated a single MEC scenario where a user has a series of dependent tasks. They transformed the joint optimization problem of the service caching placement, computation offloading, and system resource allocation (CPU processing frequency and transmit power of user) into an equivalent pure 0-1 integer linear programming for solution. In order to minimize the computation and delay costs, Zhang et al. [15] investigated the joint allocation of service caching, transmission, and computing resources in a general scenario of multiple users with multiple tasks. They also considered the scenario where each user has a computation cost budget and formulated a quadratically constrained quadratic program problem solved with a semi-definite relaxation-based algorithm. However, these studies make the ideal assumption that caching and other resources can be scheduled simultaneously, which leads to frequent cache switching and high system communication overheads.

Several researchers, including Wen et al. [38] and Sun et al. [39], have addressed the issue by incorporating dual timescale into account in doing multi-dimensional resources allocation. In [38], a consensus alternating direction method of multipliers-based joint policy for the caching, offloading, and time slot allocation was proposed to minimize the weighted sum energy consumption subject to the storage and delay constraints. However, the temporal relationship between the caching and the offloading is replaced by simple statistics, meaning that the caching strategy is done based solely on the static statistics. To address this, Sun et al. [39] proposed a joint multi-agent RL and game approach to manage multi-dimensional resources in Fog radio access networks. RL is used to learn the long-term service caching strategies from historical information of user requests and channel states in small timescales. However, due to the inherent limitations of RL, the temporal relationship between the caching and the offloading is still not fully utilized. In addition, these works generally focus on the resource allocation of a single ES, while ignoring the collaboration of multiple ESs. Therefore, how to design an intelligent two-timescale multi-dimensional resources allocation scheme with collaborative ESs remains a challenging problem. To address this problem, we establish a collaborative MEC-enabled IIoT system, and propose a DRL-based two-timescale multi-dimensional resources scheduling scheme, DGL-DDPG.

III. System Model and Problem Formulation

A. Network Architecture

As shown in Fig. 1, the collaborative MEC network comprises of one cloud server (CS), M ESs, and N TDs. Each TD can connect with one and only one ES via a wireless link at a time, and the ES can communicate with other ES and CS through wired links. The ES m can act as an access point (AP) with a built-in MEC server, which has limited available spectrum, computing, and caching resources, represented by

Authorized licensed use limited to: Univ of Science and Tech Beijing. Downloaded on June 09,2026 at 10:45:34 UTC from IEEE Xplore. Restrictions apply.

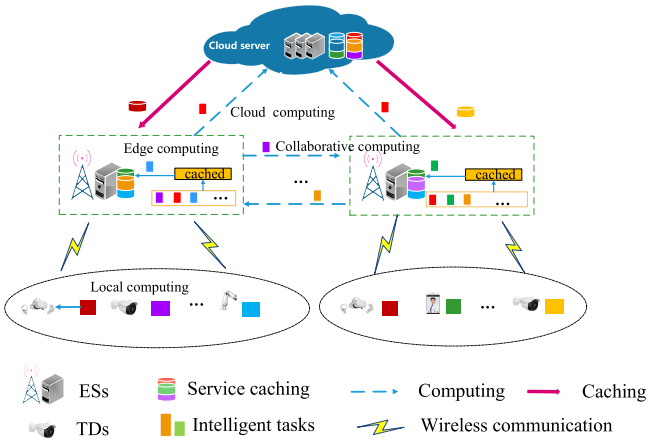


Fig. 1. Illustration of service caching, communication and computing in a collaborative MEC-enabled IIoT system.

$\{W_m, C_m, F_m\}$. We assume that all ESs can be regarded as collaborative servers and are willing to share their computing and caching resources with each other. There are F types of computing tasks and corresponding F types of service models. The service set is denoted as $\mathcal{F} = \{1, 2, \cdots, f, \cdots, F\}$. We adopts a discrete time model which is divided into two-timescale, i.e., large-timescale for service caching placement and small-timescale for task ofoading and resource allocation as shown in Fig.2. The large-timescale index of caching is denoted by $i \in I = \{1, 2, \ldots, I\}$, each of which includes T short timescale slots. The small-timescale index within the ith large-timescale slot is represented by $k \in K = \{1, 2, \ldots, K\}$. At each short time slot k, TD n randomly generates a computation task. We use a three-tuple $\{s^{type}_{n,i,k}, s^{input}_{n,i,k}, s^{comp}_{n,i,k}\}$ to denote the task of the nth TD, where $s^{type}_{n,i,k} \in \mathcal{F}$ represents the type of the task, $s^{input}_{n,i,k}$ represents the amount of input data (in bits), and $s^{comp}_{n,i,k}$ represents the required computational capacity (in CPU cycles). AS illustrated in Fig. 1, the tasks of TDs can be processed locally (e.g., red task) or offloaded to the ESs for computing. When the task is offloaded to the associated ES, if the corresponding service is cached, the task will be executed at the ES (e.g., blue and green tasks). Alternatively, the task can be further offloaded to the other collaborative ES for computing (e.g., purple and orange tasks). If none of the collaborative ES have cached the service, the task will be executed at the CS (e.g., red tasks). At each large-timescale slot i, the ESs update service caching placement (e.g., red and orange service models).

B. Service Caching Model

We define $\mathbf{c}(i) = \{c^f_m(i) \in \{0,1\} | m \in \mathcal{M}, f \in \mathcal{F}\}$ to denote the service caching decision of all ES at large-timescale i and the cache configurations of all ES maintain unchanged for a large-timescale slot. If the ES m cached the fth service at the ith large-timescale, $c^f_m(i) = 1$, otherwise, $c^f_m(i) = 0$. The total cached services should not exceed the cache capacity of the ESs, that is

$$\sum_{f \in \mathcal{F}} c^f_m(i) s^{cache}_f \leq C_m, \forall m \in \mathcal{M}, \forall i \in \mathcal{I}, \tag{1}$$

where S^{cache}_f is the storage size of the fth service model (in bits). Services in F are all available at the CS and can be cached to the ESs via backhaul. The cache switching cost is defined as the communication delay cost for downloading the services,

$$Cost(i) = \sum_{m=1}^{M} \sum_{f=1}^{F} \mathbb{I}[(c^i_f - c^{i-1}_f) = 1] \frac{s^{cache}_f}{R^{back}}, \tag{2}$$

where $\mathbb{I}[x]$ is an indicator function that equals 1 if condition x is true and 0 otherwise, $(c^f_m(i) - c^f_m(i-1)) = 1$ indicates that the service should be obtained from the CS, R^{back} is the fixed backhaul transmission rate. Obviously, the cost is 0 at the small-timescale slots.

C. Computing Model

We define $\mathbf{d}(k) = \{d^m_{n,i}(k) \in \{0,1\} | n \in \mathcal{N}, m \in \mathcal{M} \cup \{0, C\}\}$ to be the decision variable for TDs' collaborative offloading at the kth small-timescale slot in the ith large-timescale slot, $d^m_{n,i}(k) = 1$ indicates that the nth TD offloads its task to the mth ES at time slot k, similarly, $d^0_{n,i}(k) = 1$ indicates local computing and $d^C_{n,i}(k) = 1$ indicates cloud computing. Besides, we define $\mathbf{f}(k) = \{f^m_{n,i}(k) \in [0,1] | n \in \mathcal{N}, m \in \mathcal{M}\}$ as the computing resource allocation decision, $f^m_{n,i}(k) F_m$ represents the computing frequency allocated to TD n by ES m. It should be noted that the task can only be processed at the ES if and only if the relevant service model is cached at the ES and the necessary computing resources are allocated to it too. Hence, we have

$$d^m_{n,i}(k) \leq \lceil f^m_{n,i}(k) \rceil \leq c^f_m(i), \forall n \in \mathcal{N}, \forall m \in \mathcal{M}. \tag{3}$$

According to different collaborative offloading decisions, the computing mode of TD n can be divided into four cases:

Case 1: Local computing only ie., $d^0_{n,i}(k) = 1$. The local computing delay (in seconds) is

$$T^{local}_{n,i}(k) = \frac{s^{comp}_{n,i,k}}{F^l_n}, \tag{4}$$

where F^l_n is the computing capability of TD n. The corresponding energy consumption is

$$E^{local}_{n,i}(k) = \varepsilon (F^l_n)^2 s^{comp}_{n,i,k}, \tag{5}$$

where ε is the energy coefficient depending on the chip architecture of the TD n.

Case 2: Edge computing at associated ES m. In this case, the task is offloaded to ES m and the service model of task is cached, ie., $d^m_{n,i}(k) = 1$ and $c^f_m(i) = 1$. Then the computing latency can be given as

$$T^{comp}_{n,m,i}(k) = \frac{s^{comp}_{n,i,k}}{f^m_{n,i,k} F_m}. \tag{6}$$

Case 3: Edge computing at cooperative ES $\acute{m}$. In this case, there is no ability to process the offloaded task at associated ES and the task is forwarded to the cooperative ES $\acute{m}$ for processing, where the service model of task is cached and the

Authorized licensed use limited to: Univ of Science and Tech Beijing. Downloaded on June 09,2026 at 10:45:34 UTC from IEEE Xplore. Restrictions apply.

resource is sufficient at ES $\acute{m}$, ie., $d_{n,i}^{\acute{m}}(k) = 1$ **and** $c_{\acute{m}}^{f}(i) = 1$. Then the computing latency can be express as

$$T_{n,\acute{m},i}^{comp}(k) = \frac{s_{n,i,k}^{comp}}{f_{n,i}^{\acute{m}}(k) F_{\acute{m}}}, \tag{7}$$

where $\acute{m} \in \mathcal{M}/m$ is the cooperative ES of the ES m, $f_{n,i}^{\acute{m}}(k)F_{\acute{m}}$ is the computing frequency allocated to TD n by ES $\acute{m}$.

Case 4: Could computing. In this case, the service model of task is not cached at any ES, so it can only be further offloaded to the CS for computing, ie., $d_{n,i}^{C}(k) = 1$. The computing latency can be written as

$$T_{n,C,i}^{comp}(k) = \frac{s_{n,i,k}^{comp}}{F_c}, \tag{8}$$

where F_c is the computing capability of the CS.

D. Communication Model

Considering that the data size of the task processing results is negligible compared to the data size of the tasks themselves, in this work we only focus on the uplink transmission. We consider the scenario that each ES is pre-assigned spectrum bandwidth to serve its associated TDs, and we define $\mathbf{b}(k) = \{b_{n,i}^{m}(k) \in [0,1] | n \in \mathcal{N}, m \in \mathcal{M}\}$ as the bandwidth allocation decision, $b_{n,i}^{m}(k)$ represents the bandwidth allocated to the nth TD by the mth ES for task offloading at the kth small-timescale slot within the ith large-timescale slot. The uplink transmission rate from TD n to ES m can be written as

$$R_{n,i}^{m}(k) = b_{n,i}^{m}(k) W_m log_2((1 + \frac{p_n h_{n,m,i}(k)}{\sigma^2}), \tag{9}$$

where p_n is the uplink transmit power of TD n, $h_{n,m,i}(k) = g_{n,m,i}(k) dis_{n,m}^{-\alpha}$ represents the corresponding channel gain, with $g_{n,m}(k)$ and $dis_{n,m}^{-\alpha}$ are the small-scale Rayleigh fading and the large-timescale path fading, respectively, $dis_{n,m}$ represents the distance between TD n and ES m and α is the path loss exponent of the TD-ES links, σ^2 denotes the variance of the additive white Gaussian noise. We assume that the wired links transmit data at a fixed rate R^{co} and R^{back} (in bits per second, bps). The transmission delay and corresponding energy consumption from TD n to its associated ES m for task offloading can be defined as

$$T_{n,m,i}^{trans}(k) = \frac{s_{n,i,k}^{input}}{R_{n,i}^{m}(k)}. \tag{10}$$

$$E_{n,i}^{trans}(k) = p_n \times T_{n,m,i}^{trans}(k). \tag{11}$$

The transmission delay from ES m to the cooperative ES $\acute{m}$ for task offloading is written as

$$T_{m,k,i}^{trans}(k) = \frac{s_{n,i,k}^{input}}{R^{co}}. \tag{12}$$

The transmission delay from ES m to the CS for task offloading is written as

$$T_{m,C,i}^{trans}(k) = \frac{s_{n,i,k}^{input}}{R^{back}}. \tag{13}$$

E. QoS Model

The delay for processing the task of the nth TD at the kth small-timescale slot within the ith large-timescale slot for any case can be expressed as

$$T_{n,i}(k) = \begin{cases} T_{n,i}^{local}(k), & d_{n,i}^{0}(k) = 1 \\ T_{n,m,i}^{trans}(k) + T_{n,m,i}^{comp}(k), & d_{n,i}^{m}(k) = 1 \\ T_{n,m,i}^{trans}(k) + T_{m,\acute{m},i}^{trans}(k) + T_{n,\acute{m},i}^{comp}(k), & d_{n,i}^{\acute{m}}(k) = 1 \\ T_{n,m,i}^{trans}(k) + T_{m,C,i}^{trans}(k) + T_{n,C,i}^{comp}(k), & d_{n,i}^{C}(k) = 1. \end{cases} \tag{14}$$

The energy consumption for processing the task of the nth TD at the kth small-timescale slot within the ith large-timescale slot for any case can be expressed as

$$E_{n,i}(k) = (1 - d_{n,i}^{0}(k)) E_{n,m,i}^{trans}(k) + d_{n,i}^{0}(k) E_{n,i}^{local}(k). \tag{15}$$

As stated in Section I, delay and energy consumption are critical performance metrics for TDs in IIoT systems. Therefore, we define the QoS for TDs, in terms of the satisfaction with latency and energy consumption relative to a set threshold. This can be expressed as

$$Q_{n,i}(k) = \alpha \times \frac{T_n^{th} - T_{n,i}(k)}{T_n^{th}} + \beta \times \frac{E_n^{th} - E_{n,i}(k)}{E_n^{th}}, \tag{16}$$

where $\alpha, \beta \in [0,1]$, $\alpha + \beta = 1$ are the weighted parameters specifying the TD's preferences for delay and energy consumption in the QoS, respectively, T_n^{th}, E_n^{th} are the threshold of the latency and energy, which guarantee TD n can work.

F. Problem Formulation

Our objective is to maximize the long-term QoS of all TDs, while reducing the long-term cache switching costs of all ESs in a collaborative MEC-enabled IIoT system. To achieve this goal, we propose a two-timescale multi-dimensional resource joint optimization framework by properly designing the service caching, collaborative offloading, computation, and bandwidth resource allocation. The problem of multi-dimensional resource allocation can be formulated as

$$\max_{\mathbf{c}(i), \mathbf{d}(k), \mathbf{f}(k), \mathbf{b}(k)} \quad \lim_{I \to \infty} \frac{1}{I} \sum_{i=1}^{I} [\sum_{k=1}^{K} \sum_{n=1}^{N} Q_{n,i}(k) - \delta Cost(i)] \tag{17}$$

$$\text{s.t. } d_{n,i}^{m}(K) \le \lceil f_{n,i}^{m}(K) \rceil \le c_m^f(k), \quad \forall n \in \mathcal{N}, \forall m \in \mathcal{M} \tag{17a}$$

$$b_{n,i}^{m}(k) \le (1 - d_{n,i}^{0}(k)), \forall n \in \mathcal{N}, \quad \forall m \in \mathcal{M} \tag{17b}$$

$$\sum_{m \in \mathcal{M} \cup \{0, C\}} d_{n,i}^{m}(k) = 1, \forall n \in \mathcal{N} \tag{17c}$$

$$\sum_{f \in \mathcal{F}} c_m^f(i) s_f^{cache} \le C_m, \forall m \in \mathcal{M} \tag{17d}$$

$$\sum_{n \in \mathcal{N}} d_{n,i}^{m}(k) f_{n,i}^{m}(k) \le 1, \forall m \in \mathcal{M} \tag{17e}$$

$$\sum_{n \in \mathcal{N}} b_{n,i}^{m}(k) \le 1, \forall m \in \mathcal{M} \tag{17f}$$

Authorized licensed use limited to: Univ of Science and Tech Beijing. Downloaded on June 09,2026 at 10:45:34 UTC from IEEE Xplore. Restrictions apply.

$$E_{n,i}(k) \leq E_n^{th}, \forall n \in \mathcal{N} \tag{17g}$$
$$T_{n,i}(k) \leq T_n^{th}, \forall n \in \mathcal{N} \tag{17h}$$
$$c_m^f(i) \in \{0,1\}, \forall m \in \mathcal{M}, \forall f \in \mathcal{F} \tag{17i}$$
$$d_{n,i}^m(k) \in \{0,1\}, \forall n \in \mathcal{N}, \forall m \in \mathcal{M} \cup \{0, C\} \tag{17j}$$
$$f_{n,i}^m(k) \in [0,1], \forall n \in \mathcal{N}, \forall m \in \mathcal{M} \tag{17k}$$
$$b_{n,i}^m(k) \in [0,1], \forall n \in \mathcal{N}, \forall m \in \mathcal{M} \tag{17l}$$

where δ is a factor balancing the weight of the cache switching cost and the TDs'QoS, the constraint (17a) indicates that the task cannot be processed if there is no relevant service model cached at the ES or no computing resources have been allocated to it, Constraint (17b) implies that the bandwidth of ESs is allocated only to the TDs offloading their tasks, Constraint (17c) indicates that the task generated by TD n can only be processed by a single server, Constraints (17d, 17e, 17f) denote the limitations of the caching, computing, and bandwidth capacities of each ES, respectively, Constraints (17g, 17h) indicate the maximum delay and energy limitations of TDs. Constraints (17i, 17j, 17k, 17l) make the problem a mixed-integer nonlinear programming (MINLP) problem, with variables on different timescales, which typically results in an NP-hard problem. The challenge lies in the difficulty of modeling the temporal relationship between the service caching and the task offloading. The scale and complexity of the problem are prohibitively high for utilizing traditional optimization methods to solve it. To address this issue, we propose a two-timescale solution DGL-DDPG in the next section.

IV. DRL-Based Two-Timescale Resource Allocation

In this section, we first introduce the overall framework of the designed two-timescale multi-dimensional algorithm DGL-DDPG, and then we introduce the large-timescale LSTM-DDPG-based service caching algorithm and the Improved-GA-based collaborative offloading, computing and bandwidth resource allocation algorithm for the small timescale. Finally, the complexity of the proposed DGL-DDPG is analyzed.

A. The Framework of the DGL-DDPG-Based Two-Timescale Algorithm

As illustrated in Fig. 2, the DGL-DDPG algorithm consists of two layers, one is the large-timescale LSTM-DDPG based service caching, the other is the small-timescale Improved-GA based Off-RA. Specifically, service caching decisions are made at the beginning of each large-timescale slot i by an LSTM-DDPG agent, while each large-timescale caching slot is composed of K small-timescale Off-RA slots. The Improved-GA is used to make the corresponding Off-RA decisions in each small-timescale slot k. In particular, Fig. 2 shows the case that a large-timescale slot includes three small-timescale slots. First of all, the LSTM-DDPG agent observes the states of the environment over the first three small-timescale slots and feeds them into the LSTM network to learn, and then the LSTM network outputs an encoded state, that is, the current state of the large-timescale slot. It is then fed into DDPG network to get the large-timescale service caching decision. Finally, the caching decision is executed in the environment, the small-timescale Improved-GA will continue to output the Off-RA decisions and update the states of the environment under the new caching decision, and the LSTM-DDPG agent will continue to collect states and update the caching decision at the next large-timescale slot. The core of the two-layer optimization is to use the LSTM network to predict the time-varying cache requirements on a small timescale and, similar to the iterative algorithm, the two layers of DGL-DDPG are trained together until convergence. Correspondingly, as shown in Fig. 3, the input of the small-timescale Improved-GA algorithm is the service caching decision, and the output is the Off-RA decisions.

B. LSTM-DDPG Based Service Caching Strategy

The variant Off-RA decisions affect the large-timescale service caching decisions, and the current decisions will also affect the future decisions. For example, the ESs tend to store those services that are frequently used but not cached. So, this is a typical dynamic sequential decision problem. Based on this, the dynamic environment and original optimization problem can be transformed into a Markov decision process (MDP) across different timescales and then we use deep reinforcement learning algorithm to solve it. The reason for choosing LSTM-DDPG as the DRL agent mainly because of its capabilities of model-free temporal decision making and state prediction. Specifically, DDPG is a state-of-the-art DRL algorithm based on stable and efficient actor-critic framework. When the LSTM network is embedded in DDPG as the temporal feature extraction network future caching demands can be predicted. In the following, we define the state, action and reward function of service caching and then the design of LSTM-DDPG agent was presented.

1) Service caching MDP. **State** Since the service caching strategy relies heavily on the small-timescale Off-RA decisions and collaboration between ESs. The state space consists of the current service caching configuration of each ESs, the cumulative cache gain of each service, and the popularity of each service on each ES and is written as $\mathbf{s}_k = \{\mathbf{c}_k, \mathbf{g}_k, \mathbf{p}_k\}$, with

$$\mathbf{c}_k = \{c_m^f(k) | m \in \mathcal{M}, f \in \mathcal{F}\}_{M \times F}, \tag{18}$$
$$\mathbf{g}_k = \{g_f(k) | f \in \mathcal{F}\}_{1 \times F}, \tag{19}$$

where $g_f(k)$ describes the cumulative cache gain of service f, $g_f(k) = \sum_{\tau \in I} \sum_{k \in K} \sum_{n \in N} \mathbb{I}[s_{n,\tau,k}^{type} = f] Q_{n,\tau}(k) - Q_{n,\tau}^l(k)$, and $Q_{n,\tau}(k) - Q_{n,\tau}^l(k)$ represents the QoS gain caused by edge service caching compared to local computing, $Q_{n,\tau}^l(k) = \alpha \times \frac{T_{n,\tau}^{th}(k) - T_{n,\tau}^l(k)}{T_{n,\tau}^{th}(k)} + \beta \times \frac{E_{n,\tau}^{th}(k) - E_{n,\tau}^l(k)}{E_{n,\tau}^{th}(k)}$ denotes the Qos of TD n when computing locally.

$$\mathbf{p}_k = \{p_{m,f}(k) | m \in \mathcal{M}, f \in \mathcal{F}\}_{M \times F}. \tag{20}$$

where $p_{m,f}(k) = \sum_{\tau \in I} \sum_{k \in K} \sum_{n \in N} \mathbb{I}[s_{n,\tau,k}^{type} = f] d_{n,\tau}^m(k)$ represents the popularity of service f in ES m.

To predict the time-varying service caching demands, the agent will observe the state once in each small-timescale slot k, and then use the LSTM network to transform all the small

Authorized licensed use limited to: Univ of Science and Tech Beijing. Downloaded on June 09,2026 at 10:45:34 UTC from IEEE Xplore. Restrictions apply.

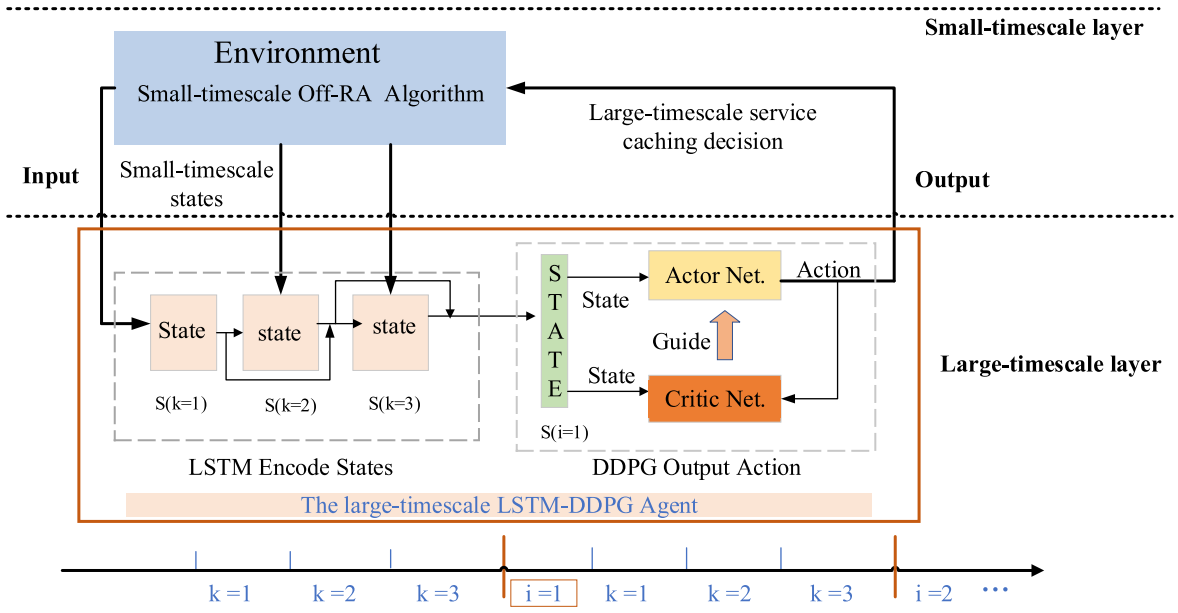


Fig. 2. The framework of the DGL-DDPG.

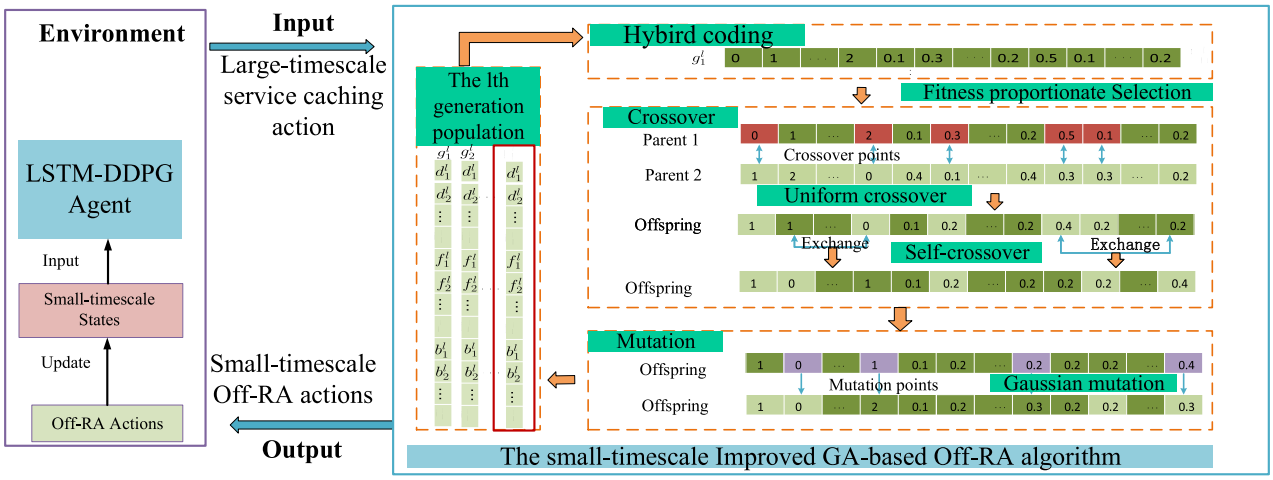


Fig. 3. The framework of the small-timescale Improved GA based Off-RA.

time slot states$\{s_1, \cdots, s_k, \cdots, s_T\}$ into a large-timescale state S_i, which will be feed into DDPG network to output the large-timescale service caching decision.

$$S_i = LSTM(s_1, \cdots, s_k, \cdots, s_T) \tag{21}$$

Action: The action is the service caching policy of each ES, $\mathbf{a}_i = \{\mathbf{c}_1(i), \cdots, \mathbf{c}_M(i)\}_{M\times F}, \mathbf{c}_m(i) = \{c_m^1(i), \cdots, c_m^f(i), \cdots, c_m^F(i)\}$.

Reward: For better caching strategy, we take into account the long-term QoS of TDs, the cache switching cost and the penalty for exceeding the cache space. The reward function can be described as follows.

$$r_i = r_{qos}(i) - r_{cost}(i) - r_{penalty}(i). \tag{22}$$

$$r_{qos}(i) = \sum_{k=1}^{K}\sum_{n=1}^{N} Q_{n,i}(k). \tag{23}$$

$$r_{cost}(i) = \sum_{m=1}^{M}\sum_{f=1}^{F} \mathbb{I}[c_m^f(k) - c_m^f(k-1) = 1]\frac{s_f^{cache}}{R^{back}}. \tag{24}$$

$$r_{penalty}(i) = \sum_{m=1}^{M} \min\{(\sum_{f=1}^{F} c_m^f(k) s_f^{cache} - C_m), 0\}. \tag{25}$$

2) The design of LSTM-DDPG. As illustrated in Fig. 2, LSTM-DDPG comprises three main components: the actor network, the critic network, and an episode experience pool. Both the actor and critic networks have a target network and a current network that have exactly the same structure. Each neural network is comprised of an input layer, output layer, two hidden layers, and a LSTM layers.

The actor network is responsible for generating the cache action according to the observed state.

$$\mathbf{a}_i = \mu(\mathbf{s}_i|\boldsymbol{\theta}^a) + \eta^i. \tag{26}$$

Authorized licensed use limited to: Univ of Science and Tech Beijing. Downloaded on June 09,2026 at 10:45:34 UTC from IEEE Xplore. Restrictions apply.

where η^i is a random Gaussian variable that decreases with the training steps to allow the agent to explore potential better caching strategy.

The primary function of the critic network is to assess the actions performed by the actor network. This is accomplished by learning a state-action value function $\mathcal{Q}(\mathbf{s}, \mathbf{a}|\boldsymbol{\theta}^c) = E\{\sum_{i=0}^{\infty} \gamma \times r_i\}$ that can be computed recursively using the Bellman equation,

$$\mathcal{Q}(\mathbf{s}_i, \mathbf{a}_i|\boldsymbol{\theta}^c) = E\{r_i(\mathbf{s}_i, \mathbf{a}_i) + \gamma E\{\mathcal{Q}(\mathbf{s}_{i+1}, \mathbf{a}_{i+1}|\boldsymbol{\theta}^c)\}\}, \tag{27}$$

where γ represents the discount factor utilized to indicate the extent of future impact exerted by the current actions.

The experience replay technique involves sampling a mini-batch of transitions from the episode experience pool. Using these sampled transitions, the actor applies the policy gradient to update the current network parameters $\boldsymbol{\theta}^a$

$$\nabla_{\boldsymbol{\theta}^a} J(\boldsymbol{\theta}^a) \approx E[\nabla_{\boldsymbol{\theta}^a} \mathcal{Q}(\mathbf{s}_i, \mathbf{a}_i|\boldsymbol{\theta}^c) \nabla_{\boldsymbol{\theta}^a} \mu(\mathbf{s}_i, \boldsymbol{\theta}^a)], \tag{28}$$

and the critic adjusts parameter $\boldsymbol{\theta}^c$ by minimizing the loss function

$$Loss(\boldsymbol{\theta}^c) = E[(\mathcal{Y} - \mathcal{Q}(\mathbf{s}_i, \mathbf{a}_i|\boldsymbol{\theta}^c))^2], \tag{29}$$

and

$$\mathcal{Y} = r_i + \gamma \acute{\mathcal{Q}}(\mathbf{s}_{i+1}, \acute{\mu}(\mathbf{s}_{i+1}|\acute{\boldsymbol{\theta}^a}), \acute{\boldsymbol{\theta}^c}), \tag{30}$$

where LSTM-DDPG utilizes the two network technology to improve the stabilization of the actor and critic, $\mathcal{Y}$ and $\acute{\mu}(\mathbf{s}_{\tau+1}|\acute{\boldsymbol{\theta}^a})$ are the output of the target network of actor and critic, respectively.

In addition, the parameters of the target networks $\acute{\boldsymbol{\theta}^a}, \acute{\boldsymbol{\theta}^c}$ can be softly updated according to the real-time updated parameter $\boldsymbol{\theta}^a$ and $\boldsymbol{\theta}^c$, respectively,

$$\boldsymbol{\theta}^{a'} \leftarrow \zeta \boldsymbol{\theta}^a + (1 - \zeta)\boldsymbol{\theta}^{a'}, \tag{31}$$

$$\boldsymbol{\theta}^{c'} \leftarrow \zeta \boldsymbol{\theta}^c + (1 - \zeta)\boldsymbol{\theta}^{c'}, \tag{32}$$

where ζ is the soft update factor. The details of LSTM-DDPG are shown in Algorithm 1.

Algorithm 1 Training Process of the LSTM-DDPG

Input : The training episode numbers S, the training steps $I \times K$, the discount factor γ, the soft update factor ζ, the replay buffer D, the mini-batch sampling size N, the critic net learning rate l^c, the actor net learning rate l^a, the Gaussian noise η^τ

Output: The actor networks' weights $\boldsymbol{\theta}^a$

```
Initialize: The actor network μ, the critic network Q with weights θ^a and θ^c;
for j ← 1 to S do
    Reset the environment, get the initial state;
    for i ← 1 to I do
        The agent makes the caching action a by observing the state s, executes the action;
        for k ← 1 to K do
            Obtain the Off-RA decisions using Algorithm 2;
            Update the state of environment by using formulas ( 19, 20);
            if k == K − 1 then
                Get a reward r, transform to the new state ś according to formula (21);
                Store the transition (s_i^j, a_i^j, r_i^j, s_{i+1}^j) into buffer D;
            end
        end
    end
end
if j > N then
    Sample a mini-batch experiences from D;
    Update the actor network by using the policy gradient (28);
    Update the critic network by minimizing the loss function (29);
    Softly update the parameters of the target network in both the actor and the critic networks by formulas (31, 32);
end
```

C. Improved-GA Based Off-RA Strategy

If the service caching policy $\mathbf{c}_i^\star$ is given, the problem can be reduced to a series of one-shot collaborative offloading and computing and bandwidth resource allocation (Off-RA) problem

$$\max_{\mathbf{d}(k),\mathbf{b}(k),\mathbf{f}(k)} \sum_{n=1}^{N} Q_{n,i}(k) \tag{33}$$

$$\text{s.t. } d_{n,i}^m(k) \le \lceil f_{n,i}^m(k) \rceil \le c_m^{f\star}(i), \forall n \in \mathcal{N}, \forall m \in \mathcal{M} \tag{34a}$$

$$b_{n,i}^m(k) \le (1 - d_{n,i}^0(k)), \forall n \in \mathcal{N}, \forall m \in \mathcal{M} \tag{34b}$$

$$\sum_{m \in \mathcal{M} \cup \{0,C\}} d_{n,i}^m(k) = 1, \forall n \in \mathcal{N} \tag{34c}$$

$$\sum_{n \in \mathcal{N}} d_{n,i}^m(k) f_{n,i}^m(k) \le 1, \forall m \in \mathcal{M} \tag{34d}$$

$$\sum_{n \in \mathcal{N}} b_{n,i}^m(k) \le 1, \forall m \in \mathcal{M} \tag{34e}$$

$$E_{n,i}(k) \le E_n^{th}, \forall n \in \mathcal{N} \tag{34f}$$

$$T_{n,i}(k) \le T_n^{th}, \forall n \in \mathcal{N} \tag{34g}$$

$$d_{n,i}^m(k) \in \{0,1\}, \forall n \in \mathcal{N}, \forall m \in \mathcal{M} \cup \{0,C\} \tag{34h}$$

$$f_{n,i}^m(k) \in [0,1], \forall n \in \mathcal{N}, \forall m \in \mathcal{M} \tag{34i}$$

$$b_{n,i}^m(k) \in [0,1], \forall n \in \mathcal{N}, \forall m \in \mathcal{M} \tag{34j}$$

However, this is an instantaneous one-step MINLP, which cannot be converted into MDP and thus cannot be solved by RL algorithms. To solve it, we introduce the GA, which is widely used to solve single-step resource optimization problems as formulated in our small timescale slots with remarkable performance. And it can be easily extended and embedded into DRL algorithms [24]. To adopt the GA,

Authorized licensed use limited to: Univ of Science and Tech Beijing. Downloaded on June 09,2026 at 10:45:34 UTC from IEEE Xplore. Restrictions apply.

we design a hybrid coding and cross-mutation scheme for solving the problem with both integer and continuous variables and redesign the fitness function to ensure that the constraints are not violated and proposed the Improved GA. The details of the Improved-GA based Off-RA algorithm are shown in the following.

1) Chromosome and hybird coding. The chromosome expression of an individual represents a solution to the Off-RA problem, which consists of three parts. To satisfy the constraints (34h, 34i, 34j), the chromosome is encoded to a discrete integer part and two continuous real value parts. For example, we denote the chromosome of individual j in the lth generation as $g_j^l(k) = \{0, 1, \cdot, 1; 0, 0.3, \cdot, 0.2; 0, 0.2, \cdot, 0.3\}_{N\times 3}$.

2) Fitness function. The fitness function represents the quality of an individual's resource allocation strategy, with higher fitness indicating better performance. To construct the fitness function, we use the objective function of the total QoS of TDs. However, during the process of crossover and mutation, it is possible to generate individuals that violate constraints. To ensure the feasibility of the solution, we design the fitness function in a piecewise form. Specifically, if an individual violates the constraints, its fitness is defined as the minimum possible value. The fitness function is defined as follows

$$F_{g_j^l}(k) = \begin{cases} e^{-3} & violation \\ \sum_{n=1}^{N} Q_{n,i}(k) & else. \end{cases} \tag{35}$$

3) Selection, Crossover, and Mutation To preserve the excellent chromosomes in the current population, we introduce the fitness proportionate selection operation. Following the principle of survival of the fittest, certain chromosomes are selected as parent nodes for the next generation. The probability that the jth chromosome g_j^l in the lth generation is selected as a parent node defined as

$$\mathbf{PR}_{g_j^l}(k) = \frac{F_{g_j^l}(k)}{\sum_{g^l \in \mathcal{G}^l} F_{g^l}(k)}. \tag{36}$$

After selection, parent nodes generate offspring through crossover and mutation. However, the original uniform crossover operation is unsuitable for consecutively coded chromosomes. As an alternative to direct swapping, we take the mean of the parents. Furthermore, for our problem, this type of crossover operation may not fully capture the coupling between variables. To address this, we add a self-crossover operation after the original uniform crossover. This operation randomly swaps genes in each block of discrete and continuous genes separately. To obtain the final chromosome of the next generation, a mutation operation is required with a specific probability. Integer random mutation is applied to the discrete gene block, whereas real Gaussian mutation is applied to the continuous gene block. For instance, if the parents have the chromosome $\{0, 1, 2, 0.2, 0.4.0.3, 0.1, 0.2, 0.6\}$, when their $(1, 5, 7)$th genes are altered, the chromosome of offspring is $\{2, 1, 1, 0.2, 0.45.0.3, 0.05, 0.2, 0.6\}$. The detailed process is summarized in Algorithm 2, and the framework is shown in Fig. 2.

Algorithm 2 Improve GA-Based Off-RA Algorithm

Input : $c^\star$,L, P, G, p_c, p_m
Output: $g^*(k)$

Randomly generate initial population $\mathcal{G}^1(k)$;
for $l \leftarrow 1$ **to** L **do**
  for $j \leftarrow 1$ **to** $|\mathcal{G}|$ **do**
    Encode each chromosome $g_j^l(k)$ hybridly ;
    Calculate the fitness of each individual $g_j^l(k)$ using formula (35);
  end
  Select excellent parent nodes $\mathcal{P}^l(k)$ from population $\mathcal{G}^l(k)$ based on probability (36);
  for $x \leftarrow 1$ **to** P^l **do**
    Select another parent node from $\mathcal{P}^l$ randomly and perform original uniform crossover option with probability p_c;
    Perform self-crossover operation with probability p_c;
    for $d \leftarrow 1$ **to** G **do**
      Perform mutation operation in each gene with probability p_m;
    end
  end
  $\mathcal{G}^{l+1} \leftarrow \mathcal{G}^l$
end
$g^* \leftarrow Max(\mathcal{G}^L)$

D. Computational Complexity of the Proposed Algorithms

For the proposed LSTM-DDPG based service caching algorithm, the training process runs offline in the CS where there are sufficient computing resources. Therefore, we focus only on the complexity of the online service caching process. For the LSTM-DDPG agent, only the actor network works online. We take the number of parameters in the model to represent the complexity of the algorithm. The actor network has an input layer, an output layer, two hidden layers, and a LSTM layer. The complexity of the actor network can be written as $O(N_i^a L_1 + 4(L_1 L_2 + L_2^2 + L_2) + L_2 N_o^a)$, where N_i^a is the number of neurons in the input layer, N_o^a is the number of neurons in the output layer, L_1, L_2 are the number of neurons in the hidden layers.

The complexity of Algorithm 2 lies in the design of coding, crossover and mutation options, but it is more directly limited by the generations and size of the population. Suppose the population size is P and the size of the chromosome is $G = 3 \times N$ and the number of generations is L, the computational complexity is about $O(2L + 7LP + 3LPN)$.

V. Simulation Results and Analysis

To validate the efficiency of the proposed DGL-DDPG scheme, we first verify the convergence of the DGL-DDPG algorithm, and then we verify the long-term and short-term system performance of the proposed algorithm under different environment settings. To investigate the performance in detail, we define the objective function, namely the weighted sum of

Authorized licensed use limited to: Univ of Science and Tech Beijing. Downloaded on June 09,2026 at 10:45:34 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SIMULATION PARAMETERS

Parameters	Values
Number of ESs M	[2,5]
Number of TDs N	[20,50]
Input size of task s^{input}	[500,5000]KB
Cache size of task s^{cache}	[2,20]GB
Computing frequency of task	[400,1000]$cycle/bit$
Energy coefficient of TD ε	5×10^{-27}
Computing capacity of CS F^c	5GHz
Computing capacity of TD F^l	1GHz
Weight factor α, β	0.5,0.5
Balance factor δ	0.001
Size of each generation P	30
Crossover rate p_c	0.45
Mutation rate p_m	0.1
The soft update factor ζ	0.05
Actor network learning rate l_a	0.001
Critic network learning rate l_c	0.002
Discount factor γ	0.95
Memory pool capacity D	10000
Mini-batch size	32

all TD's QoS and cache switching costs of all ESs, as the long-term system performance evaluation metric U_{large}, and define the average QoS of all TDs in a small timescale slot as the short-term performance metric. In detail, they are expressed as

$$U_{large} = \sum_{i=1}^{I}(\sum_{k=1}^{K}\sum_{n=1}^{N} Q_{n,i}(k) - \delta Cost(i)). \tag{37}$$

$$U_{small} = \frac{1}{N}\sum_{n=1}^{N} Q_{n,i}(k). \tag{38}$$

A. System Setups

We consider a scenario consisting of one CS, M ESs, and N TDs within a 500m $\times$ 500m square area. The system has 10 types of tasks and services, each TD generates one task randomly per small-timescale slot according to a Zipf distribution with a parameter of 0.8. For each ES, the available bandwidth, computing, and caching resource are set to 20MHz, 20GHz, and 50GB, respectively. The TD's transmission power and computing capacity are set to 20dBm and 1GHz, respectively. The fixed transmission rates R^{back} and R^{co} are set to be 200Mbps and 20Mbps, respectively. The additive white Gaussian noise σ^2 is set to 114dBm, with a path loss exponent of 2. Table I summarizes the rest simulation parameters. To show the priority of DGL-DDPG algorithm, eight similar algorithms are adopted as baselines.

- TWIN-DDPG, where a DDPG agent has two actor networks, one making the large-timescale service caching decisions and the other making the small-timescale Off-RA decisions [40].
  the service caching is optimized based on the traditional DDPG algorithm, and the Off-RA strategy is determined by using the proposed Improved-GA.
- DDPG-based service caching (DDPG), where the service caching is optimized based on the traditional DDPG algorithm, and the Off-RA strategy is determined by using the proposed Improved-GA.
- Genetic algorithm (GA), where the service caching, offloading, bandwidth, and computing resource allocation decisions are generated by traditional GA. This approach employs discrete coding scheme.
- No-cooperation, the No-cooperation scheme does not consider the collaborative computing and caching among ESs. If the associated ES has related services, the TD can offload, otherwise, it will be processed locally or offloaded to the CS.
- Ave-resource, the communication and computing resources are allocated equally to TDs, while the service caching and task offloading are optimized based on our scheme.
- Popular-cache, the Popular caching scheme caches service models at ESs based on statistical popularity information, while Off-RA decisions are made based on our proposed scheme.
- Random-cache, the Random caching scheme caches service models randomly at ESs under the constraint of cache capacity, while Off-RA decisions are made based on our scheme.
- Random-off, the Random offloading scheme generates offloading decisions randomly, while the other decisions are made based on our proposed scheme.

B. The Convergence Property of DGL-DDPG Algorithm

The proposed algorithm consists of both the large-timescale LSTM-DDPG based service caching and the small-timescale Improved-GA based Off-RA. The service caching strategy is learned by the LSTM-DDPG agent, and the agent gives a service caching decision at the beginning of each large-timescale slot, and then the Off-RA policy is obtained by the Improved-GA. First of all, we make sure that the small-timescale Improved-GA based Off-RA converges for any given service caching strategy. It is then embedded into the large timescale LSTM-DDPG based service caching to train the LSTM-DDPG agent. Therefore, when the large-timescale LSTM-DDPG converges, it means that the designed two-timescale DGL-DDPG algorithm converges. So, in the following, we first analyze the convergence property of the small-timescale Improved-GA based Off-RA, and then the convergence and scalability of the large-timescale LSTM-DDPG. Finally, we compare the convergence performance of the proposed LSTM-DDPG with that of the algorithm TWIN-DDPG.

Fig. 4 shows the convergence property of the proposed small-timescale Improved-GA when there are 2 ESs and 20 to 50 TDs. In the training process of LSTM-DDPG agent, we set the iteration number to be 20 steps. It is seen that when the number of TDs is small Improved-GA converges faster, and converges within about 10 steps, when the number of TDs increases, the convergence speed of the algorithm slows down, but the optimal decision is still obtained in about 20 steps. This is because with the hybrid coding and self-crossover operations in Improved-GA, fewer exploration steps are needed for better exploration results. This also reduces the training complexity and time of the large-timescale LSTM-DDPG.

Authorized licensed use limited to: Univ of Science and Tech Beijing. Downloaded on June 09,2026 at 10:45:34 UTC from IEEE Xplore. Restrictions apply.

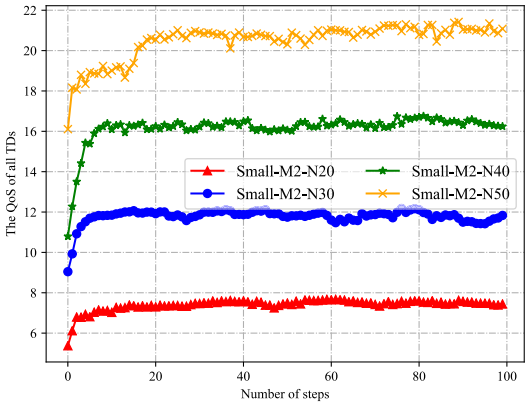


Fig. 4. Convergence performance of Improved-GA.

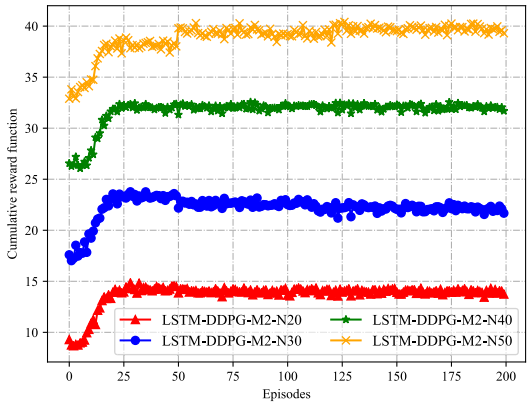


Fig. 5. Convergence performance of accumulative reward for LSTM-DDPG.

Fig. 5 shows the convergence property of the proposed large-timescale LSTM-DDPG with 2 ESs and 20 to 50 TDs. It is seen that the proposed algorithm converges well for different numbers of TDs, and its convergence is not affected by the number of TDs. This is because after the LSTM layer is added to DDPG network, the action space and state space of DDPG are independent of the number of TDs, which increases the scalability of the LSTM-DDPG.

Fig. 6 shows the cumulative reward of the proposed large-timescale LSTM-DDPG in the case of 2 to 5 ESs with 20 to 50 of TDs. We can see that as the number of ES and TD increases, the convergence speed of the algorithm slows down, but the strategy is still obtained after a certain learning rounds. The reason is that the state and action space of the agent become larger, and more experiential learning is needed to get the optimal strategy. In addition, it should be pointed out that the reason for the decrease of the cumulative reward is that the cost of cache switching in the whole system increases as the number of ES increases.

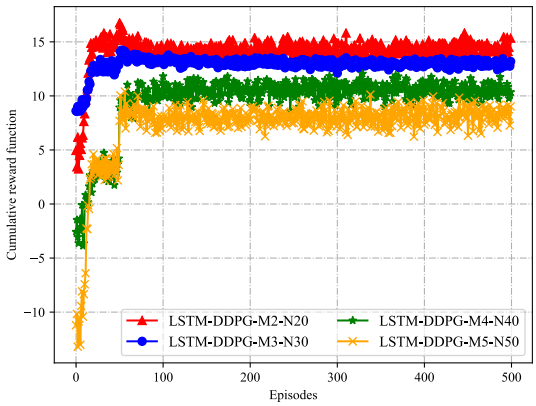


Fig. 6. Scalability performance of accumulative reward for LSTM-DDPG.

Fig. 7 shows the cumulative reward of the proposed LSTM-DDPG and TWIN-DDPG with 2 ESs and 20 TDs. It is observed that LSTM-DDPG performs better and more stable than TWIN-DDPG. In the early stages of experience accumulation and exploration, LSTM-DDPG grows rapidly and converges, while TWIN-DDPG converges slowly after several oscillations. This means that TWIN-DDPG is more difficult to train and converges slower than LSTM-DDPG. In other words, the proposed algorithm integrates traditional GA algorithm into DRL algorithm, reduces the complexity and training difficulty of LSTM-DDPG, and makes it converge faster and more stable. In addition, we observe that the performance (reward function) of the LSTM-DDPG is better. This is because compared to DDPG's network, the LSTM-based DDPG network structure can better leverage historical information to predict unknown network observation transitions. Effective predictions reduce the number of invalid explorations and therefore make it converge faster and perform better.

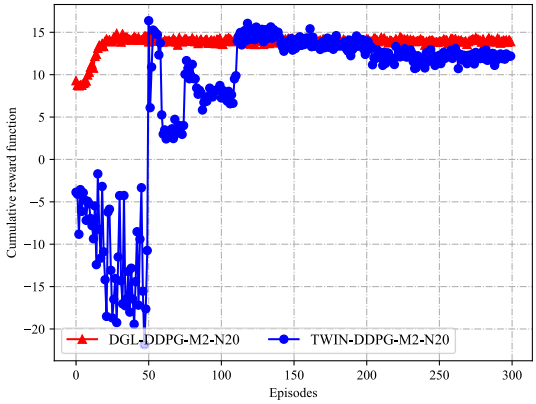


Fig. 7. The cumulative reward of the proposed LSTM-DDPG and TWIN-DDPG.

C. The Performance of DGL-DDPG at Large-Timescale

To fully verify the efficiency of DGL-DDPG in dealing with the two-timescale problem, we compare its long-term system performance, U_{large}, with that of DDPG and GA algorithm under different environment settings. The simulation is all in the setting of $I = 20, K = 5, M = 2, N = 20$ and the result takes 100 averaging operations.

Fig. 8 presents the long-term utility of DGL-DDPG compared to other benchmarks under different caching capacity settings. When the caching capacity is set to a low value (20GB-60GB), the long-term utility of DGL-DDPG is significantly better than those of GA and Popular-cache. This is because that GA only optimize single-step cache decisions based on the current offloading and resource allocation state, while Popular-cache always cache a few fixed, highly popular

Authorized licensed use limited to: Univ of Science and Tech Beijing. Downloaded on June 09,2026 at 10:45:34 UTC from IEEE Xplore. Restrictions apply.

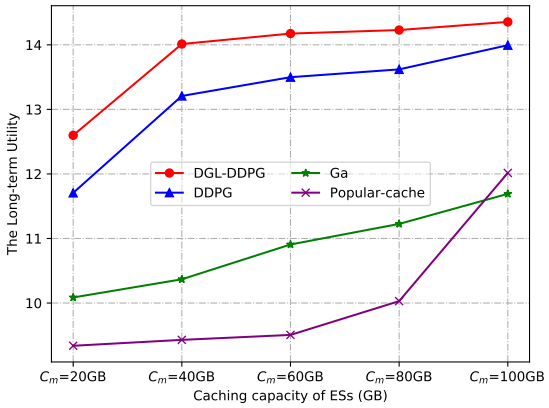


Fig. 8. The long-term utility versus the cache capacity of ESs.

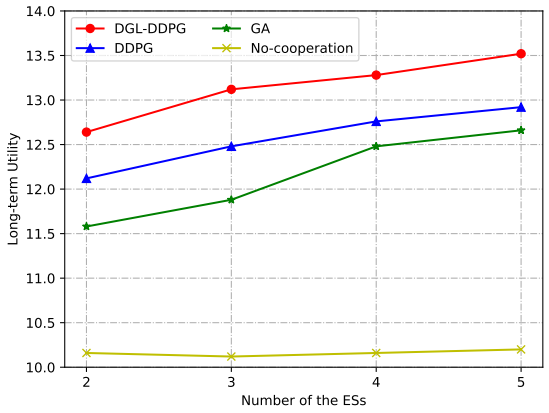


Fig. 9. The long-term utility versus the number of collaborative ES.

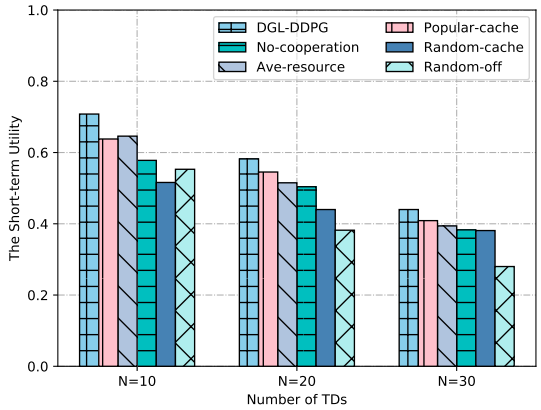


Fig. 10. The short-term utility versus the number of TDs.

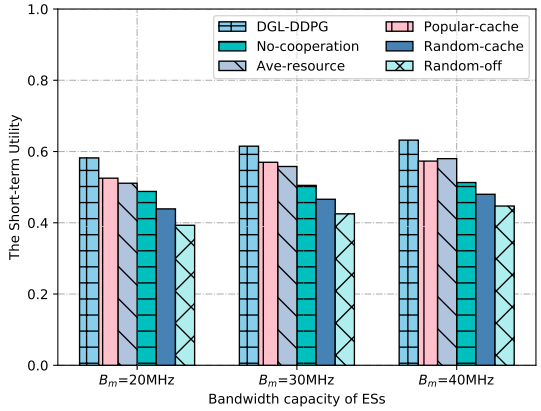


Fig. 11. The short-term utility versus the bandwidth resource of ESs.

service models. As a result, these benchmarks fail to adapt to dynamic offloading environments and result in poor long-term performance. In the same time, when achieving the same performance, DRL-based schemes occupy less caching resources than instantaneous GA and statistics-based Popular-cache. Moreover, the performance gap between DGL-DDPG and DDPG further validates the effectiveness of adopting LSTM into DDPG for the long-term service caching phase.

The long-term utility for different number of the ES is shown in Fig. 9, where No-cooperation is used as a benchmark. Apparently, the utility will increase as the number of the ESs increases. The reason is that the whole system can schedule more resources. Both DGL-DDPG and DDPG outperform GA, demonstrating the superiority of DRL in adapting to dynamic environment and solving the long-term optimization problem, In addition, that DGL-DDPG performs better than DDPG indicating that the proposed DGL-DDPG is more suited for solving the resource allocation problem over two-timescale, since the LSTM network can enhance the temporal decision-making ability of DDPG.

D. The Performance of DGL-DDPG at Small-Timescale

In this part, we compare the instantaneous performance U_{small} of the DGL-DDPG algorithm at small-timescale slot.

Fig. 10 shows the impacts of the number of TDs on the short-term utility. As the number of TDs increases, the short-term utility of all schemes continuously decreases. This is because the total resources of the ESs are limited. However, DGL-DDPG still performs better than Popular-cache and Ave-resource when the number of TD is large. The performance advantage over No-cooperation, Random-cache and Random-off is more obvious. The reason is that DGL-DDPG can achieve higher resource utilization through the form of collaboration and joint resource optimization.

The short-term utility with different bandwidth resource at the ESs is shown in Fig. 11. It is seen that the bandwidth resources increase, the short-term utility of all the simulated schemes improves, and DGL-DDPG still performs the best. This is because that the bandwidth resources limit the number of the offloaded tasks, and an increment in bandwidth resources enables the ESs to handle more tasks. Moreover, DGL-DDPG achieves a higher gain than No-cooperation and Popular-cache schemes. It reveals that even if tasks can be offloaded to the ESs, they cannot be handled if there are no the corresponding service model and computing resources, which means it is necessary to schedule the multi-dimensional resources together.

Fig. 12 shows the impacts of the computing capacity of the ESs on the short-term utility. It is seen that the short-term utility of all schemes improves as the computing capacity of the ESs increasing from 20GHz to 30 GHz. The reason is that more tasks can be processed at the ESs with the lower task processing delay and energy consumption when there is more computing resources. This reveals that DGL-DDPG

Authorized licensed use limited to: Univ of Science and Tech Beijing. Downloaded on June 09,2026 at 10:45:34 UTC from IEEE Xplore. Restrictions apply.

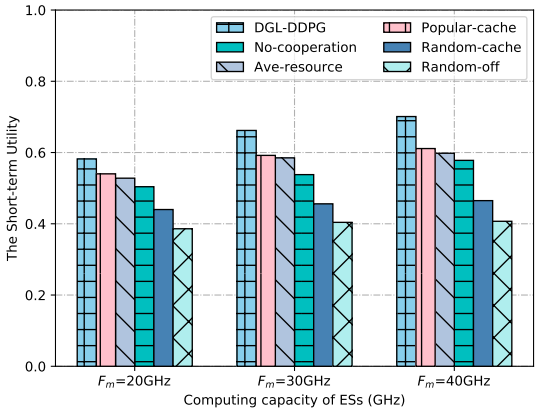


Fig. 12. The short-term utility versus the computing resource of ESs.

realizes a reasonable service caching strategy along with the offloading decisions to make full use of the computing resource. Moreover, the gain compared to No-cooperation verifies the advantage of collaboration.

VI. Conclusion

This article proposed a joint solution for service caching, collaborative offloading, computing, and bandwidth resources allocation for MEC-enabled IIoT systems with ESs cooperation. The system utility was defined by tasking account into the QoS of TDs and the cache switching costs, based on which a long-term utility maximizing problem was formulated under the constraints of limited resources. The optimization problem is two-timescale MINLP. To efficiently solve it, we proposed a two-timescale multi-dimensional resources allocation scheme, DGL-DDPG. A LSTM-DDPG agent was designed to solve the service caching for the large timescale by mining the temporal relationship between long-term service caching and short-term Off-RA decisions. Then, an Improved-GA algorithm was embedded into the agent to make instantaneous Off-RA decisions by incorporating a hybrid coding and cross-mutation schemes. Extensive simulations were conducted, and the results had demonstrated the superiority of the proposed DGL-DDPG scheme in terms of the convergence, the average QoS of TDs and the long-term utility through comparison with the other benchmarks under different system settings.

References

[1] X. Dai et al., "Task co-offloading for D2D-assisted mobile edge computing in industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 480–490, Jan. 2023.

[2] Y. Chen, Y. Sun, B. Yang, and T. Taleb, "Deep reinforcement learning-based joint caching and computing edge service placement for sensing-data-driven IIoT applications," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Seoul, South Korea, May 2022, pp. 4287–4292.

[3] T. Qiu, J. Chi, X. Zhou, Z. Ning, M. Atiquzzaman, and D. O. Wu, "Edge computing in industrial Internet of Things: Architecture, advances and challenges," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 4, pp. 2462–2488, 4th Quart., 2020.

[4] Y. Zhang and H.-Y. Wei, "Risk-aware cloud-edge computing framework for delay-sensitive industrial IoTs," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 3, pp. 2659–2671, Sep. 2021.

[5] Y. Deng, X. Chen, G. Zhu, Y. Fang, Z. Chen, and X. Deng, "Actions at the edge: Jointly optimizing the resources in multi-access edge computing," *IEEE Wireless Commun. Mag.*, vol. 29, no. 2, pp. 192–198, Apr. 2022.

[6] W. Liu, H. Zhang, H. Ding, and D. Yuan, "Delay and energy minimization for adaptive video streaming: A joint edge caching, computing and power allocation approach," *IEEE Trans. Veh. Technol.*, vol. 71, no. 9, pp. 9602–9612, Sep. 2022.

[7] Y. Zhang, B. Di, Z. Zheng, J. Lin, and L. Song, "Distributed multi-cloud multi-access edge computing by multi-agent reinforcement learning," *IEEE Trans. Wireless Commun.*, vol. 20, no. 4, pp. 2565–2578, Apr. 2021.

[8] P. Wang, B. Di, L. Song, and N. R. Jennings, "Multi-layer computation offloading in distributed heterogeneous mobile edge computing networks," *IEEE Trans. Cogn. Commun. Netw.*, vol. 8, no. 2, pp. 1301–1315, Jun. 2022.

[9] Y.-C. Wu, C. Lin, and T. Q. S. Quek, "A robust distributed hierarchical online learning approach for dynamic MEC networks," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 2, pp. 641–656, Feb. 2022.

[10] J. Yan, S. Bi, L. Duan, and Y.-J. A. Zhang, "Pricing-driven service caching and task offloading in mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 20, no. 7, pp. 4495–4512, Jul. 2021.

[11] L. Chen, G. Gong, K. Jiang, H. Zhou, and R. Chen, "DDPG-based computation offloading and service caching in mobile edge computing," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, May 2022, pp. 1–6.

[12] X.-Q. Pham, T.-D. Nguyen, V. Nguyen, and E.-N. Huh, "Joint service caching and task offloading in multi-access edge computing: A QoE-based utility optimization approach," *IEEE Commun. Lett.*, vol. 25, no. 3, pp. 965–969, Mar. 2021.

[13] S.-W. Ko, S. J. Kim, H. Jung, and S. W. Choi, "Computation offloading and service caching for mobile edge computing under personalized service preference," *IEEE Trans. Wireless Commun.*, vol. 21, no. 8, pp. 6568–6583, Aug. 2022.

[14] Y. Zhang, M. A. Kishk, and M.-S. Alouini, "Computation offloading and service caching in heterogeneous MEC wireless networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 6, pp. 3241–3256, Jun. 2023.

[15] G. Zhang, S. Zhang, W. Zhang, Z. Shen, and L. Wang, "Joint service caching, computation offloading and resource allocation in mobile edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 20, no. 8, pp. 5288–5300, Aug. 2021.

[16] I. Alqerm and J. Pan, "DeepEdge: A new QoE-based resource allocation framework using deep reinforcement learning for future heterogeneous edge-IoT applications," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 4, pp. 3942–3954, Dec. 2021.

[17] K. Kamran, E. Yeh, and Q. Ma, "DECO: Joint computation scheduling, caching, and communication in data-intensive computing networks," *IEEE/ACM Trans. Netw.*, vol. 30, no. 3, pp. 1058–1072, Jun. 2022.

[18] C. Kai, H. Zhou, Y. Yi, and W. Huang, "Collaborative cloud-edge-end task offloading in mobile-edge computing networks with limited communication capability," *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 2, pp. 624–634, Jun. 2021.

[19] C. Sun, X. Wu, X. Li, Q. Fan, J. Wen, and V. C. M. Leung, "Cooperative computation offloading for multi-access edge computing in 6G mobile networks via soft actor critic," *IEEE Trans. Netw. Sci. Eng.*, early access, Apr. 30, 2021, doi: 10.1109/TNSE.2021.3076795.

[20] Y. Sahni, J. Cao, L. Yang, and Y. Ji, "Multi-hop multi-task partial computation offloading in collaborative edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 5, pp. 1133–1145, May 2021.

[21] L. Tan, Z. Kuang, J. Gao, and L. Zhao, "Energy-efficient collaborative multi-access edge computing via deep reinforcement learning," *IEEE Trans. Ind. Informat.*, vol. 19, no. 6, pp. 7689–7699, Jun. 2023.

[22] Q. Chen, Z. Kuang, and L. Zhao, "Multiuser computation offloading and resource allocation for cloud-edge heterogeneous network," *IEEE Internet Things J.*, vol. 9, no. 5, pp. 3799–3811, Mar. 2022.

[23] B. Li, X. Deng, X. Chen, Y. Deng, and J. Yin, "MEC-based dynamic controller placement in SD-IoV: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 71, no. 9, pp. 10044–10058, Sep. 2022.

[24] H. Zhang, Y. Yang, B. Shang, and P. Zhang, "Joint resource allocation and multi-part collaborative task offloading in MEC systems," *IEEE Trans. Veh. Technol.*, vol. 71, no. 8, pp. 8877–8890, Aug. 2022.

[25] J. Zhang, S. Chen, X. Wang, and Y. Zhu, "Dynamic reservation of edge servers via deep reinforcement learning for connected vehicles," *IEEE Trans. Mobile Comput.*, vol. 22, no. 5, pp. 2661–2674, May 2023.

[26] D. Li, H. Zhang, D. Yuan, and M. Zhang, "Learning-based hierarchical edge caching for cloud-aided heterogeneous networks," *IEEE Trans. Wireless Commun.*, vol. 22, no. 3, pp. 1648–1663, Mar. 2023.

Authorized licensed use limited to: Univ of Science and Tech Beijing. Downloaded on June 09,2026 at 10:45:34 UTC from IEEE Xplore. Restrictions apply.

[27] D. Li, H. Zhang, H. Ding, T. Li, D. Liang, and D. Yuan, "User-preference-learning-based proactive edge caching for D2D-assisted wireless networks," *IEEE Internet Things J.*, vol. 10, no. 3, pp. 11922–11937, Jul. 2023.
[28] J. Zhang, S. Chen, X. Wang, and Y. Zhu, "DeepReserve: Dynamic edge server reservation for connected vehicles with deep reinforcement learning," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2021, pp. 1–10.
[29] H. Liu and G. Cao, "Deep reinforcement learning-based server selection for mobile edge computing," *IEEE Trans. Veh.*, vol. 70, no. 12, pp. 13351–13363, Dec. 2021.
[30] L. Tan, Z. Kuang, L. Zhao, and A. Liu, "Energy-efficient joint task offloading and resource allocation in OFDMA-based collaborative edge computing," *IEEE Trans. Wireless Commun.*, vol. 21, no. 3, pp. 1960–1972, Mar. 2022.
[31] Y. Deng, Z. Chen, X. Chen, and Y. Fang, "Throughput maximization for multiedge multiuser edge computing systems," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 68–79, Jan. 2022.
[32] W. Chu, P. Yu, Z. Yu, J. C. S. Lui, and Y. Lin, "Online optimal service selection, resource allocation and task offloading for multi-access edge computing: A utility-based approach," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 4150–4167, Jul. 2023.
[33] H. Zhou, Z. Zhang, Y. Wu, M. Dong, and V. C. M. Leung, "Energy efficient joint computation offloading and service caching for mobile edge computing: A deep reinforcement learning approach," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 2, pp. 950–961, Jun. 2023.
[34] H. Zhou, Z. Wang, H. Zheng, S. He, and M. Dong, "Cost minimization-oriented computation offloading and service caching in mobile cloud-edge computing: An A3C-based approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 3, pp. 1326–1338, May 2023.
[35] J. Zhang, Y. Shen, Y. Wang, X. Zhang, and J. Wang, "Dual-timescale resource allocation for collaborative service caching and computation offloading in IoT systems," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 1735–1746, Feb. 2023.
[36] R. Zhou, X. Wu, H. Tan, and R. Zhang, "Two time-scale joint service caching and task offloading for UAV-assisted mobile edge computing," in *Proc. IEEE Conf. Comput. Commun.*, May 2022, pp. 1189–1198.
[37] S. Bi, L. Huang, and Y. A. Zhang, "Joint optimization of service caching placement and computation offloading in mobile edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 19, no. 7, pp. 4947–4963, Jul. 2020.
[38] W. Wen, Y. Cui, T. Q. S. Quek, F. Zheng, and S. Jin, "Joint optimal software caching, computation offloading and communications resource allocation for mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 7, pp. 7879–7894, Jul. 2020.
[39] Y. Sun, S. Chen, Z. Wang, and S. Mao, "A joint learning and game-theoretic approach to multi-dimensional resource management in fog radio access networks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 2, pp. 2550–2563, Feb. 2023.
[40] Z. Wang, Y. Wei, F. R. Yu, and Z. Han, "Utility optimization for resource allocation in multi-access edge network slicing: A twin-actor deep deterministic policy gradient approach," *IEEE Trans. Wireless Commun.*, vol. 21, no. 8, pp. 5842–5856, Aug. 2022.



Qianqian Liu received the B.E. degree from Shandong Normal University, China, in 2021. She is currently pursuing the M.S. degree with the School of Control Science and Engineering, Shandong University, Jinan, China. Her research interests include joint caching, computing, and communication resource optimization in MEC networks.



Haixia Zhang (Senior Member, IEEE) received the B.E. degree from the Department of Communication and Information Engineering, Guilin University of Electronic Technology, Guilin, China, in 2001, and the M.Eng. and Ph.D. degrees in communication and information systems from the School of Information Science and Engineering, Shandong University, Jinan, China, in 2004 and 2008, respectively. From 2006 to 2008, she was an Academic Assistant with the Institute for Circuit and Signal Processing, Munich University of Technology, Munich, Germany. From 2016 to 2017, she was a Visiting Professor with the University of Florida, Gainesville, FL, USA. She is currently a Distinguished Professor with Shandong University. Her research interests include wireless communication and networks, the industrial Internet of Things, wireless resource management, and mobile edge computing. She is actively participating in many professional services. She serves/served as the symposium chair, a TPC member, the session chair, and a keynote speakers for many conferences. She is/was an Editor of IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS, IEEE WIRELESS COMMUNICATION LETTERS, and *China Communications*.



Xin Zhang received the B.E. degree from China University of Petroleum in 2021. He is currently pursuing the M.S. degree with the School of Control Science and Engineering, Shandong University, Jinan, China. His research interests include trajectory design and resource allocation in UAV networks.



Dongfeng Yuan (Senior Member, IEEE) received the M.S. degree from the Department of Electrical Engineering, Shandong University, Jinan, China, in 1988, and the Ph.D. degree from the Department of Electrical Engineering, Tsinghua University, Beijing, China, in January 2000. From 1993 to 1994, he was with the Electrical and Computer Department, University of Calgary, Alberta, AB, Canada. From 1998 to 1999, he was with the Department of Electrical Engineering, University of Erlangen, Erlangen, Germany. From 2001 to 2002, he was with the Department of Electrical Engineering and Computer Science, University of Michigan, Ann Arbor, MI, USA. In 2005, he was with the Department of Electrical Engineering, Munich University of Technology, Munich, Germany. In 2006, he was with the Department of Electrical Engineering, Heriot-Watt University, Edinburgh, U.K. He is currently a Full Professor with Shandong Key Laboratory of Intelligent Communication and Sensing-Computing Integration, Shandong Univeristy. His current research interests include intelligent communication systems, mobile edge computing and cloud computing, and AI and big data processing for communications.

Authorized licensed use limited to: Univ of Science and Tech Beijing. Downloaded on June 09,2026 at 10:45:34 UTC from IEEE Xplore. Restrictions apply.